AI and RAG

A 12-page practice document for PDF extraction, chunking, embeddings, retrieval, and evaluation

Designed for local experiments with Python, Ollama, and vector search.

Suggested exercise: extract this document page by page, create overlapping chunks, generate embeddings, store page metadata, and test whether natural-language questions retrieve the correct sections.

1. Large Language Model Foundations

A large language model is a neural network trained to predict tokens from context. During training, the model receives enormous collections of text and learns statistical patterns involving words, grammar, style, facts, and common reasoning structures. The model does not store a conventional database of complete sentences. Instead, information is distributed across many numerical parameters that influence which token is likely to come next.

When a user sends a prompt, the text is converted into tokens and passed through multiple transformer blocks. Each block updates the internal representation of every token by considering relationships with other tokens. The final layer produces a probability distribution over the model vocabulary. A decoding strategy chooses the next token, appends it to the sequence, and repeats the process until the answer finishes or reaches a configured limit.

Language models are strong at generating fluent text, summarizing documents, classifying content, extracting structured information, and transforming text between formats. They are not guaranteed truth engines. Their output depends on the supplied context, learned patterns, decoding settings, and application controls. Reliable systems therefore combine models with retrieval, validation, monitoring, and human review.

Key ideas include tokens, context, parameters, decoding, and uncertainty. A model can provide a useful answer without having a human-style understanding of the subject. The application surrounding the model is responsible for deciding what information may be used and what actions may be performed.

2. Tokens, Context, and Embeddings

A tokenizer divides text into smaller units called tokens. A token may represent a complete word, part of a word, punctuation, or whitespace. Tokenization allows a model to work with a finite vocabulary while still handling unfamiliar words. Context windows, usage limits, latency, and API costs are commonly measured in tokens rather than characters or words.

The context window contains all information available to the model for the current generation. It may include a system instruction, conversation history, the current user message, retrieved passages, and tool results. Information outside the context window is not directly visible to the model. Applications often shorten older conversations, summarize history, or retrieve only relevant documents.

An embedding converts text into a numerical vector. Texts with similar meanings tend to produce vectors located near each other in vector space. Embeddings are useful for semantic search because they can match meaning rather than exact keywords. A question about annual leave can retrieve a document about vacation days even if the two texts use different wording.

A retrieval system should use the same embedding model for documents and queries. Changing the model without rebuilding the index can make stored vectors incompatible with new query vectors. Metadata such as document name, page number, section, version, and access level should be stored beside every vector.

3. Transformer Blocks and Attention

The transformer is the core architecture behind most modern language models. A model contains many repeated transformer blocks. Each block normally includes self-attention, a feed-forward network, normalization, and residual connections. Repeating these operations allows the model to build increasingly useful representations of the input sequence.

Self-attention helps each token identify other relevant tokens. For every token, the model calculates query, key, and value vectors. Queries are compared with keys to produce attention weights, and those weights control how values are combined. Multi-head attention performs this process in parallel, allowing different heads to focus on different relationships.

Decoder-only language models use a causal mask. This prevents a token from attending to future tokens during generation. When predicting the next token, the model can use the prompt and all tokens already generated, but it cannot see tokens that have not yet been produced. This design matches the next-token prediction objective used during training.

Residual connections add the original input back after a transformation, which helps information and gradients travel through deep networks. Layer normalization stabilizes values. The feed-forward network transforms each token representation independently after attention has mixed contextual information.

4. Prompt Design and Conversation Roles

A prompt communicates the task, relevant context, constraints, and expected output. Clear prompts describe what the model should do, identify the audience, specify the required format, and explain what to do when information is unavailable. Ambiguous prompts force the model to infer missing requirements and usually create less predictable results.

Chat applications commonly organize input into system, user, and assistant roles. The system message defines broad behavior and boundaries. User messages contain requests and data. Assistant messages preserve previous responses so the model can continue the conversation. These roles are serialized into tokens before inference.

Zero-shot prompting gives instructions without examples. One-shot prompting provides one example, while few-shot prompting provides several examples. Examples are useful when the output format, classification labels, tone, or decision criteria are difficult to describe. They should be accurate, diverse, and consistent with the instructions.

Prompt quality can be tested systematically. Keep a fixed evaluation set, change one prompt element at a time, and compare accuracy, format compliance, latency, and token usage. A prompt that performs well on one example may fail on unusual wording, missing information, adversarial text, or long documents.

5. Hallucination and Answer Reliability

A hallucination is an unsupported or incorrect model statement presented as if it were reliable. Hallucinations occur because language models optimize next-token prediction rather than factual verification. When evidence is missing, a model may continue a plausible linguistic pattern instead of stopping. Fluent language can make an incorrect answer appear convincing.

Common causes include missing knowledge, outdated training data, ambiguous questions, conflicting context, high sampling randomness, and pressure to always provide an answer. In retrieval systems, poor document selection is another major cause. If the correct passage is not retrieved, the generation model cannot reliably use it.

Applications can reduce hallucination by supplying trustworthy context, lowering temperature for factual tasks, requesting citations, validating structured output, and allowing the model to state that information is unavailable. Critical decisions require deterministic business rules or human review rather than relying only on generated language.

Evaluation should separate retrieval quality from generation quality. Retrieval metrics determine whether the right evidence was found. Faithfulness checks determine whether the answer is supported by that evidence. Answer relevance determines whether the response addresses the question.

6. Retrieval-Augmented Generation Overview

Retrieval-augmented generation, usually called RAG, connects a language model to an external knowledge source. Instead of expecting the model to remember every fact, the application searches a document collection for relevant passages and places those passages in the prompt. The model then generates an answer using the retrieved evidence.

A RAG system has two broad phases. The indexing phase loads documents, extracts text, cleans it, divides it into chunks, creates embeddings, and stores the resulting vectors with metadata. The query phase embeds the user question, retrieves similar chunks, optionally reranks them, constructs a prompt, and asks the language model to answer.

RAG can make answers more current, domain-specific, and traceable. Updating the knowledge base is often easier than retraining a language model. It also allows answers to include references to source documents. However, RAG does not automatically guarantee correctness. Poor parsing, weak chunking, incomplete metadata, or irrelevant retrieval can still produce bad answers.

Successful RAG projects treat ingestion and retrieval as data engineering problems. They define document ownership, update schedules, versioning, quality checks, access controls, observability, and recovery procedures. A polished chatbot interface cannot compensate for an unreliable knowledge pipeline.

7. Document Ingestion and Cleaning

Document ingestion begins by collecting source files and recording where they came from. Sources may include PDFs, web pages, office documents, database rows, support tickets, or APIs. Each source should receive a stable identifier and metadata describing its owner, creation time, version, permissions, and processing status.

Text extraction is not always straightforward. Digital PDFs usually contain selectable text, while scanned PDFs contain images and require optical character recognition. Tables may lose their rows and columns during extraction. Headers, footers, page numbers, navigation elements, and repeated legal notices can pollute the content and reduce retrieval quality.

Cleaning should preserve meaning while removing noise. Useful steps include normalizing whitespace, repairing broken line endings, removing repeated headers, identifying headings, retaining page references, and detecting empty or corrupted pages. Every transformation should be reproducible so the index can be rebuilt when processing rules change.

Data quality checks can measure extraction success, text length, language, duplicate content, missing pages, and unusual character ratios. Failed documents should be quarantined for review rather than silently entering the vector index. Logging each stage makes it possible to diagnose why a later answer used incomplete evidence.

8. Chunking Strategies

Chunking divides long documents into smaller pieces that can be embedded and retrieved. Chunk size creates a tradeoff. Very small chunks may lose context and produce vague matches. Very large chunks may contain several unrelated topics, waste context-window space, and reduce retrieval precision.

Fixed-size chunking splits text after a chosen number of characters or tokens. It is simple and fast but may cut sentences or sections at awkward points. Overlap repeats part of one chunk in the next chunk, reducing the chance that important information is separated at a boundary. Excessive overlap increases storage and creates nearly duplicate search results.

Structure-aware chunking respects headings, paragraphs, lists, or document sections. Semantic chunking attempts to detect topic changes using embeddings or language models. These methods can preserve meaning better but require more processing and careful testing. The best strategy depends on document structure and user questions.

Each chunk should store metadata such as document identifier, page number, section title, chunk sequence, source version, and permissions. Chunking should be evaluated with real questions. If answers require information from several distant sections, the system may need parent-child retrieval, neighboring chunks, or query decomposition.

9. Vector Search and Similarity

After chunking, an embedding model converts each chunk into a vector. These vectors are stored in a vector index or vector database. At query time, the same embedding model converts the question into a vector. The system then searches for document vectors that are close to the query vector.

Cosine similarity compares the direction of two vectors and is widely used for semantic search. Other indexes may use dot product or Euclidean distance. The database can use approximate nearest-neighbor algorithms to search millions of vectors quickly. Approximation improves speed while accepting a small possibility of missing the exact nearest result.

Semantic similarity alone may not be sufficient. Metadata filters can restrict results to a department, customer, date range, language, or permission level. Hybrid retrieval combines embeddings with keyword search, helping with exact product names, codes, identifiers, and rare terminology. Reranking uses another model to reorder candidate passages.

Retrieval parameters include the number of candidates, similarity threshold, filters, and reranking depth. Increasing the number of chunks can improve recall but may add irrelevant context. These settings should be tuned using representative questions and known relevant passages rather than chosen only by intuition.

10. Building the Generation Prompt

After retrieval, the application combines instructions, retrieved passages, conversation history, and the current question. The prompt should clearly separate trusted application instructions from untrusted document text. Retrieved content may contain irrelevant statements or malicious instructions, so the model should treat it as evidence rather than authority.

A grounded prompt tells the model to answer from the supplied context, identify uncertainty, and avoid unsupported claims. It may require citations containing document names and page numbers. The application should generate citations from stored metadata instead of trusting the model to invent source details.

Context ordering can influence results. Highly relevant passages are often placed near the question or presented with clear labels. Duplicate chunks should be removed to preserve space. If retrieved evidence conflicts, the prompt can ask the model to describe the conflict instead of selecting a version without explanation.

Conversation history must be managed carefully. A follow-up question may depend on earlier turns, but unlimited history increases tokens and can distract retrieval. Applications commonly rewrite the latest question into a standalone search query, summarize older dialogue, or retain only turns relevant to the current topic.

11. Evaluation, Security, and Operations

RAG evaluation should include a dataset of realistic questions, expected evidence, and acceptable answers. Retrieval can be measured with recall at k, precision, and ranking metrics. Generated answers can be evaluated for relevance, faithfulness, completeness, citation accuracy, tone, and format compliance. Human review remains important.

Latency is another key measurement. Time to first token captures perceived responsiveness, while end-to-end latency measures the complete request. Retrieval time, reranking time, model generation time, and queue time should be recorded separately. Percentiles such as p50, p95, and p99 reveal slow experiences that averages can hide.

Prompt injection occurs when user content or retrieved documents attempt to override application instructions. Defenses include treating external text as untrusted, limiting tool permissions, validating tool arguments, requiring confirmation for consequential actions, filtering inaccessible documents, and keeping secrets outside prompts.

Operational systems need logging, monitoring, version control, and feedback loops. Teams should track which model, embedding model, prompt, index version, and documents produced each answer. This lineage makes failures reproducible. A mature RAG system is a governed data pipeline connected to a probabilistic generation component.